

Pretrained Models Interview Questions & Answers Cheat Sheet

Table of Contents

1. [Pretrained Models Fundamentals](#)
2. [Transfer Learning](#)
3. [Hugging Face Ecosystem](#)
4. [BERT and Transformer Models](#)
5. [GPT Models](#)
6. [Fine-tuning Strategies](#)
7. [Vision Models \(ResNet, VGG, etc.\)](#)
8. [Model Selection & Evaluation](#)
9. [Deployment & Optimization](#)
10. [Ethics & Bias](#)
11. [Practical Implementation](#)
12. [Advanced Topics](#)

Pretrained Models Fundamentals

Q1: What are pretrained models and why are they important?

Answer:

Pretrained Models: Neural networks that have been trained on large datasets and can be adapted for new tasks through transfer learning.

Key Benefits:

- **Faster Development:** Reduce training time from weeks to hours
- **Better Performance:** Leverage knowledge from large datasets
- **Resource Efficiency:** Less computational power and data required
- **Lower Barriers:** Makes advanced ML accessible to more teams

How They Work:

1. **Initial Training:** Model trained on large, general dataset
2. **Knowledge Encoding:** Learns universal features and patterns
3. **Transfer:** Apply learned features to new, related tasks

4. Fine-tuning: Adapt to specific requirements

Example:

```
# Using a pretrained model
from transformers import AutoModel, AutoTokenizer

# Load pretrained BERT model
model = AutoModel.from_pretrained('bert-base-uncased')
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

# The model already knows:
# - Language patterns
# - Word relationships
# - Grammatical structures
# - Semantic understanding
```

Q2: What's the difference between feature extraction and fine-tuning?

Answer:

Aspect	Feature Extraction	Fine-tuning
Base Model	Frozen (no weight updates)	Trainable (weights updated)
Training Speed	Fast	Slower
Data Requirements	Less data needed	More data beneficial
Computational Cost	Lower	Higher
Performance	Good for similar tasks	Better for task-specific performance

Feature Extraction:

```
import torch
import torch.nn as nn

# Load pretrained ResNet
backbone = torch.hub.load('pytorch/vision:v0.10.0', 'resnet18', pretrained=True)

# Freeze all parameters
for param in backbone.parameters():
    param.requires_grad = False

# Replace final layer
backbone.fc = nn.Linear(backbone.fc.in_features, num_classes)

# Only the final layer will be trained
```

Fine-tuning:

```
# Load pretrained model
model = torch.hub.load('pytorch/vision:v0.10.0', 'resnet18', pretrained=True)
```

```
# Replace final layer
model.fc = nn.Linear(model.fc.in_features, num_classes)

# All parameters are trainable (use lower learning rate)
optimizer = torch.optim.Adam([
    {'params': model.fc.parameters(), 'lr': 1e-3}, # New layer
    {'params': model.features.parameters(), 'lr': 1e-4} # Pretrained layers
])
```

When to Use:

- **Feature Extraction:** Small dataset, limited compute, task similar to pretraining
- **Fine-tuning:** Larger dataset, more compute available, task-specific optimization needed

Q3: How do you choose the right pretrained model?

Answer:

Selection Criteria:

1. Task Similarity:

- **Domain Match:** Vision model for images, language model for text
- **Task Type:** Classification, detection, generation, etc.

2. Model Architecture:

```
# For different tasks
models_by_task = {
    'image_classification': ['ResNet', 'EfficientNet', 'Vision Transformer'],
    'object_detection': ['YOLO', 'Faster R-CNN', 'DETR'],
    'text_classification': ['BERT', 'RoBERTa', 'DistilBERT'],
    'text_generation': ['GPT-2', 'GPT-3', 'T5'],
    'question_answering': ['BERT', 'ELECTRA', 'DeBERTa']
}
```

3. Performance vs Efficiency Trade-off:

Model Size	Parameters	Speed	Accuracy	Use Case
Small	<100M	Fast	Good	Mobile, real-time
Base	100-500M	Medium	Better	General purpose
Large	500M+	Slow	Best	Research, high accuracy

4. Available Resources:

```
# Model resource requirements
model_specs = {
    'bert-base-uncased': {'params': '110M', 'memory': '1.3GB', 'inference': 'fast'},
    'bert-large-uncased': {'params': '340M', 'memory': '2.7GB', 'inference': 'medium'},
}
```

```
'gpt-3': {'params': '175B', 'memory': '700GB', 'inference': 'slow'}
}
```

Selection Process:

1. **Define Requirements:** Accuracy, speed, memory constraints
2. **Survey Options:** Check model hubs (Hugging Face, PyTorch Hub)
3. **Benchmark:** Test top candidates on validation set
4. **Consider Licensing:** Commercial vs research use

Transfer Learning

Q4: Explain the concept of transfer learning and its types.

Answer:

Transfer Learning: Technique where a model trained on one task is adapted for a related task.

Types of Transfer Learning:

1. Inductive Transfer Learning:

- **Task:** Different but related
- **Domain:** Same or different
- **Labels:** Available for target task

```
# Example: ImageNet → Medical Images
source_model = torchvision.models.resnet50(pretrained=True)
# Adapt for medical image classification
source_model.fc = nn.Linear(2048, 5) # 5 medical conditions
```

2. Transductive Transfer Learning:

- **Task:** Same
- **Domain:** Different
- **Labels:** Not available for target

3. Unsupervised Transfer Learning:

- **Task:** Different but related
- **Domain:** Different
- **Labels:** Not available for target

Transfer Learning Strategies:

Layer Freezing Strategy:

```
def setup_transfer_learning(model, num_classes, freeze_layers=True):
    if freeze_layers:
        # Freeze feature extractor
        for param in model.features.parameters():
            param.requires_grad = False

        # Replace classifier
        model.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(model.classifier[1].in_features, num_classes)
        )

    return model

# Progressive unfreezing
def unfreeze_layers(model, layer_names):
    for name, param in model.named_parameters():
        if any(layer in name for layer in layer_names):
            param.requires_grad = True
```

Q5: What are the challenges in transfer learning?

Answer:

Major Challenges:

1. Negative Transfer:

- When source and target domains are too different
- Pretrained features hurt performance

```
# Detecting negative transfer
def evaluate_transfer_benefit(model_pretrained, model_scratch, test_loader):
    acc_pretrained = evaluate_model(model_pretrained, test_loader)
    acc_scratch = evaluate_model(model_scratch, test_loader)

    if acc_pretrained < acc_scratch:
        print("Negative transfer detected!")
        return False
    return True
```

2. Domain Shift:

- Distribution differences between source and target
- **Covariate Shift:** $P(X)$ changes but $P(Y|X)$ stays same
- **Concept Drift:** $P(Y|X)$ changes

3. Catastrophic Forgetting:

- Model forgets source task knowledge during fine-tuning

Solutions:

```

# Learning rate scheduling
def get_transfer_optimizer(model, base_lr=1e-4):
    return torch.optim.Adam([
        {'params': model.features.parameters(), 'lr': base_lr * 0.1},
        {'params': model.classifier.parameters(), 'lr': base_lr}
    ])

# Gradual unfreezing
def gradual_unfreezing(model, epochs_per_stage=5):
    stages = [
        ['classifier'],
        ['classifier', 'layer4'],
        ['classifier', 'layer4', 'layer3'],
        # ... continue for all layers
    ]

    for stage, layers_to_unfreeze in enumerate(stages):
        unfreeze_layers(model, layers_to_unfreeze)
        train_model(model, epochs_per_stage)

```

4. Model Selection Difficulty:

- Many available models
- Unclear which performs best for specific task

Best Practices:

- Start with models trained on similar domains
- Use validation set to detect negative transfer
- Apply progressive fine-tuning
- Monitor for catastrophic forgetting

Hugging Face Ecosystem

Q6: What is Hugging Face and why is it popular?

Answer:

Hugging Face: Open-source platform providing state-of-the-art ML models, datasets, and tools.

Key Components:

1. Transformers Library:

```

from transformers import pipeline, AutoModel, AutoTokenizer

# Easy-to-use pipelines
classifier = pipeline("sentiment-analysis")
result = classifier("Hugging Face is amazing!")

```

```
# Manual model loading
model = AutoModel.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

2. Hub (Model Repository):

- 100,000+ pretrained models
- Version control for models
- Easy sharing and collaboration

3. Datasets Library:

```
from datasets import load_dataset

# Load popular datasets
dataset = load_dataset("squad") # Stanford Question Answering
imdb = load_dataset("imdb")     # Movie reviews
```

4. Accelerate Library:

```
from accelerate import Accelerator

accelerator = Accelerator()
model, optimizer, dataloader = accelerator.prepare(
    model, optimizer, dataloader
)
```

Why Popular:

- **Ease of Use:** 3-line code for SOTA models
- **Unified Interface:** Same API for all models
- **Community:** Large, active community
- **Performance:** State-of-the-art results
- **Documentation:** Excellent tutorials and docs

Q7: How do you fine-tune a model using Hugging Face?

Answer:

Complete Fine-tuning Example:

1. Setup and Data Preparation:

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
```

```

        DataCollatorWithPadding
    )
    from datasets import Dataset
    import torch

    # Load model and tokenizer
    model_name = "bert-base-uncased"
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=2
    )
    tokenizer = AutoTokenizer.from_pretrained(model_name)

    # Prepare dataset
    def tokenize_function(examples):
        return tokenizer(
            examples["text"],
            padding="max_length",
            truncation=True,
            max_length=512
        )

    # Apply tokenization
    train_dataset = train_dataset.map(tokenize_function, batched=True)
    eval_dataset = eval_dataset.map(tokenize_function, batched=True)

```

2. Training Configuration:

```

training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="eval_accuracy",
    greater_is_better=True,
)

# Data collator
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

```

3. Training:

```

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = predictions.argmax(axis=-1)
    return {"accuracy": (predictions == labels).mean()}

```



```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

# Start training
trainer.train()

# Save model
trainer.save_model("./fine-tuned-model")

```

4. Inference:

```

# Load fine-tuned model
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="./fine-tuned-model",
    tokenizer=tokenizer
)

# Make predictions
result = classifier("This movie is fantastic!")
print(result) # [{'label': 'POSITIVE', 'score': 0.9998}]

```

Q8: What are Hugging Face pipelines and when should you use them?

Answer:

Pipelines: High-level API that groups tokenizer, model, and post-processing for common tasks.

Available Pipelines:

1. Text Tasks:

```

# Sentiment Analysis
sentiment = pipeline("sentiment-analysis")
result = sentiment("I love this product!")

# Text Generation
generator = pipeline("text-generation", model="gpt2")
text = generator("The future of AI is", max_length=50)

# Question Answering
qa = pipeline("question-answering")
result = qa(
    question="What is the capital of France?",

```

```

    context="France is a country in Europe. Its capital is Paris."
)

# Named Entity Recognition
ner = pipeline("ner", aggregation_strategy="simple")
entities = ner("My name is John and I live in New York")

# Text Summarization
summarizer = pipeline("summarization")
summary = summarizer(long_text, max_length=100)

```

2. Vision Tasks:

```

# Image Classification
classifier = pipeline("image-classification")
result = classifier("path/to/image.jpg")

# Object Detection
detector = pipeline("object-detection")
objects = detector("path/to/image.jpg")

```

3. Audio Tasks:

```

# Speech Recognition
asr = pipeline("automatic-speech-recognition")
text = asr("path/to/audio.wav")

# Audio Classification
audio_classifier = pipeline("audio-classification")
result = audio_classifier("path/to/audio.wav")

```

Custom Pipeline:

```

# Create custom pipeline
classifier = pipeline(
    "text-classification",
    model="your-custom-model",
    tokenizer="your-custom-tokenizer",
    device=0 # Use GPU
)

# With specific model configuration
sentiment = pipeline(
    "sentiment-analysis",
    model="nlp-town/bert-base-multilingual-uncased-sentiment",
    return_all_scores=True
)

```

When to Use Pipelines:

- **Prototyping:** Quick testing and experimentation

- **Simple Applications:** Standard tasks without customization
- **Demos:** Easy to showcase model capabilities
- **Batch Processing:** Process multiple inputs efficiently

When NOT to Use:

- **Custom Preprocessing:** Need specific tokenization
- **Performance Critical:** Need maximum speed optimization
- **Complex Logic:** Multi-step processing required

BERT and Transformer Models

Q9: Explain BERT architecture and its key innovations.

Answer:

BERT (Bidirectional Encoder Representations from Transformers): Transformer-based model that processes text bidirectionally.

Key Innovations:

1. Bidirectional Context:

```
# Traditional (unidirectional)
# "The cat sat on the [MASK]" → only sees left context

# BERT (bidirectional)
# "The cat sat on the [MASK]" → sees both left AND right context
```

2. Masked Language Modeling (MLM):

```
# Training objective
input_text = "The cat [MASK] on the mat"
# Model learns to predict: "sat"

# 15% of tokens are masked:
# - 80% → [MASK] token
# - 10% → random token
# - 10% → unchanged (but still predicted)
```

3. Next Sentence Prediction (NSP):

```
# Training pairs
sentence_a = "I went to the store."
sentence_b = "I bought some milk." # IsNext = True
```

```
sentence_a = "I went to the store."  
sentence_b = "The cat is sleeping." # IsNext = False
```

Architecture Details:

Input Representation:

```
# Token embeddings + Position embeddings + Segment embeddings  
class BERTEmbeddings:  
    def __init__(self, vocab_size, hidden_size, max_position):  
        self.word_embeddings = nn.Embedding(vocab_size, hidden_size)  
        self.position_embeddings = nn.Embedding(max_position, hidden_size)  
        self.token_type_embeddings = nn.Embedding(2, hidden_size)  
  
    def forward(self, input_ids, token_type_ids, position_ids):  
        word_emb = self.word_embeddings(input_ids)  
        pos_emb = self.position_embeddings(position_ids)  
        type_emb = self.token_type_embeddings(token_type_ids)  
  
        return word_emb + pos_emb + type_emb
```

Model Variants:

- **BERT-Base:** 12 layers, 768 hidden, 12 attention heads, 110M parameters
- **BERT-Large:** 24 layers, 1024 hidden, 16 attention heads, 340M parameters

Q10: How does BERT differ from GPT?

Answer:

Fundamental Differences:

Aspect	BERT	GPT
Direction	Bidirectional	Unidirectional (left-to-right)
Architecture	Encoder-only	Decoder-only
Training Objective	MLM + NSP	Autoregressive Language Modeling
Best For	Understanding tasks	Generation tasks
Input Processing	Sees entire sequence	Sequential processing

Training Objectives:

BERT (Masked Language Modeling):

```
# Input: "The [MASK] is running fast"  
# Task: Predict masked token using bidirectional context  
# Answer: "dog", "car", "horse", etc.  
  
def bert_mlm_loss(logits, masked_tokens, labels):
```

```
# Only compute loss on masked positions
loss_fct = nn.CrossEntropyLoss()
masked_logits = logits[mask_tokens]
masked_labels = labels[mask_tokens]
return loss_fct(masked_logits, masked_labels)
```

GPT (Causal Language Modeling):

```
# Input: "The dog is running"
# Task: Predict next token given previous tokens
# Sequence: "The" → "dog", "The dog" → "is", etc.

def gpt_causal_loss(logits, labels):
    # Shift labels for next token prediction
    shift_logits = logits[..., :-1, :].contiguous()
    shift_labels = labels[..., 1:].contiguous()

    loss_fct = nn.CrossEntropyLoss()
    return loss_fct(shift_logits.view(-1, shift_logits.size(-1)),
                    shift_labels.view(-1))
```

Use Cases:

BERT Strengths:

- Text classification
- Named entity recognition
- Question answering
- Sentiment analysis
- Text similarity

GPT Strengths:

- Text generation
- Creative writing
- Code generation
- Conversational AI
- Completion tasks

Implementation Example:

```
# BERT for classification
from transformers import BertForSequenceClassification
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')

# GPT for generation
from transformers import GPT2LMHeadModel
model = GPT2LMHeadModel.from_pretrained('gpt2')
```

```
# Different attention patterns
# BERT: All tokens can attend to all tokens
# GPT: Token can only attend to previous tokens (causal mask)
```

Q11: What are the different BERT variants and their use cases?

Answer:

Major BERT Variants:

1. RoBERTa (Robustly Optimized BERT):

```
from transformers import RobertaTokenizer, RobertaForSequenceClassification

# Improvements over BERT:
# - No Next Sentence Prediction
# - Dynamic masking
# - Larger batch sizes
# - More training data

model = RobertaForSequenceClassification.from_pretrained('roberta-base')
tokenizer = RobertaTokenizer.from_pretrained('roberta-base')
```

2. DistilBERT:

```
# 60% smaller, 60% faster than BERT-base
# Retains 97% of BERT's performance

from transformers import DistilBertModel
model = DistilBertModel.from_pretrained('distilbert-base-uncased')

# Use case: Mobile applications, real-time inference
```

3. ALBERT (A Lite BERT):

```
# Parameter sharing and factorized embeddings
# 18x fewer parameters than BERT-large

from transformers import AlbertModel
model = AlbertModel.from_pretrained('albert-base-v2')
```

4. ELECTRA:

```
# Replaced Token Detection instead of MLM
# More efficient pretraining

from transformers import ElectraModel
model = ElectraModel.from_pretrained('electra-base-discriminator')
```

Domain-Specific Variants:

1. SciBERT (Scientific Domain):

```
# Trained on scientific papers
from transformers import AutoModel
model = AutoModel.from_pretrained('allenai/scibert_scivocab_uncased')
```

2. BioBERT (Biomedical Domain):

```
# Trained on PubMed abstracts and full-text articles
model = AutoModel.from_pretrained('dmis-lab/biobert-v1.1')
```

3. FinBERT (Financial Domain):

```
# Fine-tuned on financial text
model = AutoModel.from_pretrained('ProsusAI/finbert')
```

Multilingual Variants:

1. mBERT (Multilingual BERT):

```
# Trained on 104 languages
model = AutoModel.from_pretrained('bert-base-multilingual-cased')
```

2. XLM-RoBERTa:

```
# Better multilingual performance
model = AutoModel.from_pretrained('xlm-roberta-base')
```

Selection Guide:

```
model_selection = {
    'high_accuracy': 'roberta-large',
    'fast_inference': 'distilbert-base-uncased',
    'memory_efficient': 'albert-base-v2',
    'scientific_text': 'allenai/scibert_scivocab_uncased',
    'multilingual': 'xlm-roberta-base',
    'general_purpose': 'bert-base-uncased'
}
```

GPT Models

Q12: Explain the evolution from GPT-1 to GPT-3 and beyond.

Answer:

GPT Evolution Timeline:

GPT-1 (2018):

- **Parameters:** 117M
- **Innovation:** Unsupervised pretraining + supervised fine-tuning
- **Architecture:** 12-layer transformer decoder

```
# GPT-1 approach
# 1. Pretrain on large text corpus (unsupervised)
# 2. Fine-tune on specific task (supervised)

def gpt1_training():
    # Stage 1: Language modeling pretraining
    pretrain_loss = language_modeling_loss(text_corpus)

    # Stage 2: Task-specific fine-tuning
    finetune_loss = classification_loss(task_data)
```

GPT-2 (2019):

- **Parameters:** 1.5B (largest variant)
- **Innovation:** Zero-shot task performance
- **Key Finding:** Scale improves capabilities

```
# GPT-2 showed emergent abilities
model = GPT2LMHeadModel.from_pretrained('gpt2-large')

# Zero-shot capabilities
prompt = "Translate English to French: Hello → "
# Model generates: "Bonjour" without translation training
```

GPT-3 (2020):

- **Parameters:** 175B
- **Innovation:** Few-shot in-context learning
- **Capabilities:** Reasoning, coding, creative writing

```
# Few-shot learning pattern
prompt = """
Q: What is the capital of France?
A: Paris

Q: What is the capital of Spain?
A: Madrid
```



```
Q: What is the capital of Italy?
```

```
A: ""
```

```
# GPT-3 generates: "Rome"
```

GPT-4 (2023):

- **Innovation:** Multimodal (text + images)
- **Improvements:** Better reasoning, reduced hallucinations
- **Capabilities:** Complex problem solving

Key Scaling Laws:

```
# Scaling relationships discovered
def compute_optimal_model_size(compute_budget):
    # Chinchilla scaling law
    optimal_params = compute_budget ** 0.5
    optimal_tokens = compute_budget ** 0.5
    return optimal_params, optimal_tokens
```

Q13: How do you implement text generation with GPT models?

Answer:

Basic Text Generation:

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch

# Load model and tokenizer
model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

def generate_text(prompt, max_length=100, temperature=1.0, top_p=0.9):
    # Encode input
    input_ids = tokenizer.encode(prompt, return_tensors='pt')

    # Generate
    with torch.no_grad():
        output = model.generate(
            input_ids,
            max_length=max_length,
            temperature=temperature,
            top_p=top_p,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    # Decode output
    generated_text = tokenizer.decode(output[0], skip_special_tokens=True)
    return generated_text

# Usage
```

```
prompt = "The future of artificial intelligence is"
generated = generate_text(prompt)
print(generated)
```

Advanced Generation Strategies:

1. Temperature Sampling:

```
def temperature_sampling(logits, temperature=1.0):
    """
    Higher temperature = more random
    Lower temperature = more focused
    """
    logits = logits / temperature
    probs = torch.softmax(logits, dim=-1)
    return torch.multinomial(probs, 1)
```

2. Top-k Sampling:

```
def top_k_sampling(logits, k=50):
    """Only sample from top-k most likely tokens"""
    top_k_logits, top_k_indices = torch.topk(logits, k)
    probs = torch.softmax(top_k_logits, dim=-1)
    chosen = torch.multinomial(probs, 1)
    return top_k_indices[chosen]
```

3. Top-p (Nucleus) Sampling:

```
def top_p_sampling(logits, p=0.9):
    """Sample from tokens that comprise top-p probability mass"""
    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    cumulative_probs = torch.cumsum(torch.softmax(sorted_logits, dim=-1), dim=-1)

    # Find cutoff
    cutoff = torch.where(cumulative_probs > p)[0][0]
    top_p_logits = sorted_logits[:cutoff+1]
    top_p_indices = sorted_indices[:cutoff+1]

    probs = torch.softmax(top_p_logits, dim=-1)
    chosen = torch.multinomial(probs, 1)
    return top_p_indices[chosen]
```

Conditional Generation:

```
class ConditionalGPT:
    def __init__(self, model_name='gpt2'):
        self.model = GPT2LMHeadModel.from_pretrained(model_name)
        self.tokenizer = GPT2Tokenizer.from_pretrained(model_name)
        self.tokenizer.pad_token = self.tokenizer.eos_token

    def generate_with_constraints(self, prompt, banned_words=None,
```

```

        required_words=None, max_length=100):
    input_ids = self.tokenizer.encode(prompt, return_tensors='pt')

    generated = input_ids
    for _ in range(max_length - input_ids.size(1)):
        with torch.no_grad():
            outputs = self.model(generated)
            logits = outputs.logits[0, -1, :]

            # Apply constraints
            if banned_words:
                for word in banned_words:
                    word_id = self.tokenizer.encode(word)[0]
                    logits[word_id] = float('-inf')

            # Sample next token
            next_token = torch.multinomial(
                torch.softmax(logits, dim=-1), 1
            )
            generated = torch.cat([generated, next_token.unsqueeze(0)], dim=1)

            if next_token == self.tokenizer.eos_token_id:
                break

    return self.tokenizer.decode(generated[0], skip_special_tokens=True)

```

Fine-tuning Strategies

Q14: What are the different fine-tuning approaches for pretrained models?

Answer:

Fine-tuning Strategies:

1. Full Fine-tuning:

```

# Update all model parameters
def full_finetuning(model, train_loader, epochs=3, lr=2e-5):
    # All parameters are trainable
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr)

    for epoch in range(epochs):
        for batch in train_loader:
            optimizer.zero_grad()

            outputs = model(**batch)
            loss = outputs.loss
            loss.backward()

            optimizer.step()

    return model

```

2. Layer-wise Fine-tuning:

```
def layer_wise_finetuning(model, train_loader):
    # Fine-tune layers progressively
    layers_to_unfreeze = [
        ['classifier'],                # Stage 1
        ['classifier', 'pooler'],      # Stage 2
        ['classifier', 'pooler', 'encoder.layer.11'], # Stage 3
        # ... continue for more layers
    ]

    for stage, layer_names in enumerate(layers_to_unfreeze):
        # Freeze all parameters
        for param in model.parameters():
            param.requires_grad = False

        # Unfreeze specified layers
        for name, param in model.named_parameters():
            if any(layer in name for layer in layer_names):
                param.requires_grad = True

        # Train current stage
        train_stage(model, train_loader, epochs=2)
```

3. Parameter-Efficient Fine-tuning:

LoRA (Low-Rank Adaptation):

```
import torch.nn as nn

class LoRALayer(nn.Module):
    def __init__(self, original_layer, rank=4, alpha=1):
        super().__init__()
        self.original_layer = original_layer
        self.rank = rank
        self.alpha = alpha

        # Freeze original weights
        for param in self.original_layer.parameters():
            param.requires_grad = False

        # Add LoRA weights
        self.lora_A = nn.Parameter(torch.randn(rank, original_layer.in_features))
        self.lora_B = nn.Parameter(torch.zeros(original_layer.out_features, rank))

    def forward(self, x):
        original_output = self.original_layer(x)
        lora_output = (x @ self.lora_A.T @ self.lora_B.T) * (self.alpha / self.rank)
        return original_output + lora_output

# Apply LoRA to linear layers
def apply_lora(model, rank=4):
    for name, module in model.named_children():
        if isinstance(module, nn.Linear):
```

```

        setattr(model, name, LoRALayer(module, rank=rank))
    else:
        apply_lora(module, rank)

```

4. Adapter-based Fine-tuning:

```

class AdapterLayer(nn.Module):
    def __init__(self, hidden_size, adapter_size=64):
        super().__init__()
        self.adapter_down = nn.Linear(hidden_size, adapter_size)
        self.adapter_up = nn.Linear(adapter_size, hidden_size)
        self.activation = nn.ReLU()

    def forward(self, x):
        # Residual connection with adapter
        adapter_output = self.adapter_up(
            self.activation(self.adapter_down(x))
        )
        return x + adapter_output

# Insert adapters into transformer layers
def add_adapters(transformer_layer, adapter_size=64):
    hidden_size = transformer_layer.attention.self.query.in_features
    adapter = AdapterLayer(hidden_size, adapter_size)

    # Insert after self-attention
    original_forward = transformer_layer.forward

    def new_forward(hidden_states, *args, **kwargs):
        outputs = original_forward(hidden_states, *args, **kwargs)
        hidden_states = adapter(outputs[0])
        return (hidden_states,) + outputs[1:]

    transformer_layer.forward = new_forward

```

Q15: How do you prevent overfitting during fine-tuning?

Answer:

Overfitting Prevention Strategies:

1. Early Stopping:

```

class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.001):
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = float('inf')
        self.patience_counter = 0

    def __call__(self, val_loss):
        if val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss

```

```

        self.patience_counter = 0
        return False
    else:
        self.patience_counter += 1
        return self.patience_counter >= self.patience

# Usage in training loop
early_stopping = EarlyStopping(patience=3)

for epoch in range(max_epochs):
    train_loss = train_epoch(model, train_loader)
    val_loss = evaluate(model, val_loader)

    if early_stopping(val_loss):
        print(f"Early stopping at epoch {epoch}")
        break

```

2. Learning Rate Scheduling:

```

from torch.optim.lr_scheduler import CosineAnnealingLR, StepLR

# Cosine annealing
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
scheduler = CosineAnnealingLR(optimizer, T_max=num_epochs)

# Step decay
scheduler = StepLR(optimizer, step_size=10, gamma=0.1)

# Warmup + decay
def get_linear_schedule_with_warmup(optimizer, warmup_steps, total_steps):
    def lr_lambda(step):
        if step < warmup_steps:
            return step / warmup_steps
        return max(0.0, (total_steps - step) / (total_steps - warmup_steps))

    return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

```

3. Regularization Techniques:

```

# Weight decay
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=2e-5,
    weight_decay=0.01 # L2 regularization
)

# Dropout
class RegularizedBERT(nn.Module):
    def __init__(self, base_model, dropout_rate=0.3):
        super().__init__()
        self.bert = base_model
        self.dropout = nn.Dropout(dropout_rate)
        self.classifier = nn.Linear(base_model.config.hidden_size, num_classes)

```

```

def forward(self, **inputs):
    outputs = self.bert(**inputs)
    pooled_output = self.dropout(outputs.pooler_output)
    return self.classifier(pooled_output)

```

4. Data Augmentation:

```

# Text augmentation techniques
def augment_text(text, augmentation_prob=0.1):
    words = text.split()
    augmented_words = []

    for word in words:
        if random.random() < augmentation_prob:
            # Synonym replacement
            augmented_words.append(get_synonym(word))
        else:
            augmented_words.append(word)

    return ' '.join(augmented_words)

# Back-translation augmentation
def back_translate(text, intermediate_language='fr'):
    # English → French → English
    fr_text = translate(text, src='en', dest=intermediate_language)
    back_text = translate(fr_text, src=intermediate_language, dest='en')
    return back_text

```

5. Cross-validation:

```

from sklearn.model_selection import StratifiedKFold

def cross_validate_model(data, labels, k_folds=5):
    kfold = StratifiedKFold(n_splits=k_folds, shuffle=True, random_state=42)
    cv_scores = []

    for fold, (train_idx, val_idx) in enumerate(kfold.split(data, labels)):
        # Split data
        train_data = [data[i] for i in train_idx]
        val_data = [data[i] for i in val_idx]

        # Train model
        model = create_model()
        train_model(model, train_data)

        # Evaluate
        score = evaluate_model(model, val_data)
        cv_scores.append(score)

    print(f"Fold {fold + 1}: {score:.4f}")

    print(f"Average CV Score: {np.mean(cv_scores):.4f} ± {np.std(cv_scores):.4f}")
    return cv_scores

```

Vision Models (ResNet, VGG, etc.)

Q16: Compare different CNN architectures (AlexNet, VGG, ResNet, EfficientNet).

Answer:

CNN Architecture Evolution:

Model	Year	Key Innovation	Parameters	ImageNet Top-1
AlexNet	2012	Deep CNN, ReLU, Dropout	61M	57.1%
VGG-16	2014	Small filters (3×3)	138M	71.3%
ResNet-50	2015	Skip connections	26M	76.2%
EfficientNet-B0	2019	Compound scaling	5.3M	77.1%

Detailed Comparison:

1. AlexNet (2012):

```
# Revolutionary for its time
import torch.nn as nn

class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            # ... more layers
        )

        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )
```

2. VGG (2014):

```
# Key insight: Stack small filters instead of large ones
class VGG16(nn.Module):
```



```

def __init__(self, num_classes=1000):
    super().__init__()
    self.features = self._make_layers([64, 64, 'M', 128, 128, 'M',
                                         256, 256, 256, 'M', 512, 512, 512, 'M',
                                         512, 512, 512, 'M'])

def _make_layers(self, cfg):
    layers = []
    in_channels = 3
    for x in cfg:
        if x == 'M':
            layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
        else:
            layers += [nn.Conv2d(in_channels, x, kernel_size=3, padding=1),
                       nn.ReLU(inplace=True)]
            in_channels = x
    return nn.Sequential(*layers)

```

3. ResNet (2015):

```

# Skip connections solve vanishing gradient
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # Skip connection
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        residual = x
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(residual) # Skip connection
        return torch.relu(out)

```

4. EfficientNet (2019):

```

# Compound scaling: depth × width × resolution
class EfficientNet(nn.Module):
    def __init__(self, width_coeff, depth_coeff, resolution, num_classes=1000):
        super().__init__()
        # Scale network dimensions
        self.width_coeff = width_coeff
        self.depth_coeff = depth_coeff

```

```

        # Mobile inverted bottleneck blocks
        self.blocks = self._make_blocks()

    def _scale_width(self, width):
        return int(width * self.width_coeff)

    def _scale_depth(self, depth):
        return int(math.ceil(depth * self.depth_coeff))

```

Transfer Learning Implementation:

```

def load_pretrained_cnn(architecture, num_classes, pretrained=True):
    models = {
        'alexnet': torchvision.models.alexnet,
        'vgg16': torchvision.models.vgg16,
        'resnet50': torchvision.models.resnet50,
        'efficientnet_b0': torchvision.models.efficientnet_b0
    }

    model = models[architecture](pretrained=pretrained)

    # Modify final layer for target classes
    if architecture in ['alexnet', 'vgg16']:
        model.classifier[-1] = nn.Linear(model.classifier[-1].in_features, num_classes)
    elif 'resnet' in architecture:
        model.fc = nn.Linear(model.fc.in_features, num_classes)
    elif 'efficientnet' in architecture:
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, num_classes)

    return model

```

Q17: How do you use pretrained CNN models for custom tasks?

Answer:

Complete Transfer Learning Pipeline:

1. Model Setup:

```

import torch
import torch.nn as nn
import torchvision.transforms as transforms
from torchvision import models

class CustomCNN(nn.Module):
    def __init__(self, num_classes, architecture='resnet50', freeze_features=True):
        super().__init__()

        # Load pretrained model
        if architecture == 'resnet50':
            self.backbone = models.resnet50(pretrained=True)
            feature_dim = self.backbone.fc.in_features
            self.backbone.fc = nn.Identity() # Remove final layer

```

```

elif architecture == 'efficientnet_b0':
    self.backbone = models.efficientnet_b0(pretrained=True)
    feature_dim = self.backbone.classifier[1].in_features
    self.backbone.classifier = nn.Identity()

# Freeze feature extractor if needed
if freeze_features:
    for param in self.backbone.parameters():
        param.requires_grad = False

# Custom classifier
self.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(feature_dim, 512),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(512, num_classes)
)

def forward(self, x):
    features = self.backbone(x)
    return self.classifier(features)

```

2. Data Preprocessing:

```

def get_transforms(input_size=224):
    # Match ImageNet preprocessing for pretrained models
    train_transforms = transforms.Compose([
        transforms.Resize((input_size + 32, input_size + 32)),
        transforms.RandomCrop(input_size),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(degrees=10),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225]) # ImageNet stats
    ])

    val_transforms = transforms.Compose([
        transforms.Resize((input_size, input_size)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225])
    ])

    return train_transforms, val_transforms

```

3. Training Strategy:

```

def train_custom_model(model, train_loader, val_loader, num_epochs=10):
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model.to(device)

```

```

# Different learning rates for different parts
optimizer = torch.optim.AdamW([
    {'params': model.backbone.parameters(), 'lr': 1e-5}, # Lower LR for pretrained
    {'params': model.classifier.parameters(), 'lr': 1e-3} # Higher LR for new layers
], weight_decay=1e-4)

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)
criterion = nn.CrossEntropyLoss()

best_acc = 0.0

for epoch in range(num_epochs):
    # Training phase
    model.train()
    running_loss = 0.0
    correct_predictions = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        outputs = model(data)
        loss = criterion(outputs, target)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        correct_predictions += (predicted == target).sum().item()

    # Validation phase
    val_acc = validate_model(model, val_loader, device)

    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pth')

    scheduler.step()

    print(f'Epoch {epoch+1}/{num_epochs}:')
    print(f'Train Loss: {running_loss/len(train_loader):.4f}')
    print(f'Val Accuracy: {val_acc:.4f}')

def validate_model(model, val_loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for data, target in val_loader:
            data, target = data.to(device), target.to(device)
            outputs = model(data)
            _, predicted = torch.max(outputs.data, 1)
            total += target.size(0)
            correct += (predicted == target).sum().item()

```

```
return correct / total
```

4. Progressive Unfreezing:

```
def progressive_unfreezing(model, train_loader, val_loader):  
    """  
    Gradually unfreeze layers during training  
    """  
    # Stage 1: Train only classifier  
    print("Stage 1: Training classifier only")  
    for param in model.backbone.parameters():  
        param.requires_grad = False  
    train_model_stage(model, train_loader, val_loader, epochs=5, lr=1e-3)  
  
    # Stage 2: Unfreeze last residual block  
    print("Stage 2: Unfreezing last block")  
    for param in model.backbone.layer4.parameters():  
        param.requires_grad = True  
    train_model_stage(model, train_loader, val_loader, epochs=3, lr=1e-4)  
  
    # Stage 3: Unfreeze all layers  
    print("Stage 3: Fine-tuning all layers")  
    for param in model.backbone.parameters():  
        param.requires_grad = True  
    train_model_stage(model, train_loader, val_loader, epochs=2, lr=1e-5)
```

Model Selection & Evaluation

Q18: How do you evaluate pretrained models for your specific task?

Answer:

Evaluation Framework:

1. Define Evaluation Metrics:

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix  
import numpy as np  
  
class ModelEvaluator:  
    def __init__(self, task_type='classification'):  
        self.task_type = task_type  
        self.metrics = {}  
  
    def evaluate_classification(self, y_true, y_pred, y_prob=None):  
        # Basic metrics  
        accuracy = accuracy_score(y_true, y_pred)  
        precision, recall, f1, _ = precision_recall_fscore_support(  
            y_true, y_pred, average='weighted'  
        )
```

```

# Confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Per-class metrics
per_class_metrics = precision_recall_fscore_support(
    y_true, y_pred, average=None
)

results = {
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1,
    'confusion_matrix': cm,
    'per_class_precision': per_class_metrics[0],
    'per_class_recall': per_class_metrics[1],
    'per_class_f1': per_class_metrics[2]
}

# AUC-ROC if probabilities available
if y_prob is not None:
    from sklearn.metrics import roc_auc_score
    try:
        auc = roc_auc_score(y_true, y_prob, multi_class='ovr')
        results['auc'] = auc
    except ValueError:
        pass # Binary classification with single class

return results

```

2. Cross-validation for Robust Evaluation:

```

def evaluate_model_cv(model_class, data, labels, cv_folds=5, **model_kwargs):
    from sklearn.model_selection import StratifiedKFold

    kfold = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=42)
    cv_results = []

    for fold, (train_idx, val_idx) in enumerate(kfold.split(data, labels)):
        print(f"Training fold {fold + 1}/{cv_folds}")

        # Split data
        train_data = [data[i] for i in train_idx]
        val_data = [data[i] for i in val_idx]
        train_labels = [labels[i] for i in train_idx]
        val_labels = [labels[i] for i in val_idx]

        # Initialize model
        model = model_class(**model_kwargs)

        # Train model
        model.fit(train_data, train_labels)

        # Evaluate

```

```

        predictions = model.predict(val_data)
        probabilities = model.predict_proba(val_data) if hasattr(model, 'predict_proba')

        fold_results = evaluate_classification(val_labels, predictions, probabilities)
        cv_results.append(fold_results)

# Aggregate results
aggregated_results = {}
for metric in cv_results[0].keys():
    if metric != 'confusion_matrix':
        values = [result[metric] for result in cv_results]
        aggregated_results[metric] = {
            'mean': np.mean(values),
            'std': np.std(values),
            'values': values
        }

return aggregated_results

```

3. Model Comparison Framework:

```

def compare_models(models_configs, train_data, test_data):
    """
    Compare multiple pretrained models on same task
    """
    results = {}

    for model_name, config in models_configs.items():
        print(f"Evaluating {model_name}...")

        # Load and setup model
        model = load_pretrained_model(config)

        # Fine-tune on training data
        start_time = time.time()
        model = finetune_model(model, train_data, config['training_params'])
        training_time = time.time() - start_time

        # Evaluate on test data
        start_time = time.time()
        test_predictions = model.predict(test_data['inputs'])
        inference_time = (time.time() - start_time) / len(test_data['inputs'])

        # Calculate metrics
        metrics = evaluate_classification(test_data['labels'], test_predictions)

        # Model complexity
        num_params = sum(p.numel() for p in model.parameters())
        model_size_mb = num_params * 4 / (1024 * 1024) # Assuming float32

        results[model_name] = {
            'metrics': metrics,
            'training_time': training_time,
            'inference_time': inference_time,
            'num_parameters': num_params,

```

```

        'model_size_mb': model_size_mb,
        'config': config
    }

    return results

# Example usage
models_to_compare = {
    'distilbert': {
        'model_name': 'distilbert-base-uncased',
        'training_params': {'lr': 2e-5, 'epochs': 3}
    },
    'roberta': {
        'model_name': 'roberta-base',
        'training_params': {'lr': 1e-5, 'epochs': 3}
    },
    'bert': {
        'model_name': 'bert-base-uncased',
        'training_params': {'lr': 2e-5, 'epochs': 3}
    }
}

```

4. Performance vs Efficiency Analysis:

```

def plot_efficiency_frontier(results):
    import matplotlib.pyplot as plt

    models = list(results.keys())
    accuracies = [results[m]['metrics']['accuracy'] for m in models]
    inference_times = [results[m]['inference_time'] for m in models]
    model_sizes = [results[m]['model_size_mb'] for m in models]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # Accuracy vs Inference Time
    ax1.scatter(inference_times, accuracies, s=[s*10 for s in model_sizes], alpha=0.7)
    for i, model in enumerate(models):
        ax1.annotate(model, (inference_times[i], accuracies[i]))
    ax1.set_xlabel('Inference Time (ms)')
    ax1.set_ylabel('Accuracy')
    ax1.set_title('Accuracy vs Speed')

    # Accuracy vs Model Size
    ax2.scatter(model_sizes, accuracies, c=inference_times, cmap='viridis')
    for i, model in enumerate(models):
        ax2.annotate(model, (model_sizes[i], accuracies[i]))
    ax2.set_xlabel('Model Size (MB)')
    ax2.set_ylabel('Accuracy')
    ax2.set_title('Accuracy vs Size')

    plt.tight_layout()
    plt.show()

```


Q19: How do you benchmark pretrained models?

Answer:

Comprehensive Benchmarking Framework:

1. Performance Benchmarking:

```
import time
import psutil
import torch
from memory_profiler import profile

class ModelBenchmark:
    def __init__(self, model, device='cuda'):
        self.model = model
        self.device = device
        self.model.to(device)

    def benchmark_inference(self, test_inputs, num_runs=100, warmup_runs=10):
        """Benchmark inference speed and memory usage"""
        self.model.eval()

        # Warmup
        for _ in range(warmup_runs):
            with torch.no_grad():
                _ = self.model(test_inputs[:1])

        torch.cuda.synchronize() if self.device == 'cuda' else None

        # Benchmark
        start_time = time.time()
        memory_before = torch.cuda.memory_allocated() if self.device == 'cuda' else 0

        for _ in range(num_runs):
            with torch.no_grad():
                outputs = self.model(test_inputs)

        torch.cuda.synchronize() if self.device == 'cuda' else None
        end_time = time.time()
        memory_after = torch.cuda.memory_allocated() if self.device == 'cuda' else 0

        # Calculate metrics
        total_time = end_time - start_time
        avg_time_per_batch = total_time / num_runs
        throughput = len(test_inputs) * num_runs / total_time
        memory_usage = (memory_after - memory_before) / (1024**2)  # MB

        return {
            'avg_inference_time': avg_time_per_batch,
            'throughput_samples_per_sec': throughput,
            'memory_usage_mb': memory_usage,
            'total_time': total_time
        }
```

```

def benchmark_training(self, train_loader, num_epochs=1):
    """Benchmark training speed"""
    self.model.train()
    optimizer = torch.optim.Adam(self.model.parameters())
    criterion = torch.nn.CrossEntropyLoss()

    start_time = time.time()
    total_samples = 0

    for epoch in range(num_epochs):
        for batch_idx, (data, target) in enumerate(train_loader):
            data, target = data.to(self.device), target.to(self.device)

            optimizer.zero_grad()
            output = self.model(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()

            total_samples += len(data)

            if batch_idx >= 10: # Limit for benchmarking
                break

    end_time = time.time()
    training_time = end_time - start_time
    samples_per_sec = total_samples / training_time

    return {
        'training_samples_per_sec': samples_per_sec,
        'total_training_time': training_time
    }

def profile_model_size(self):
    """Profile model parameters and size"""
    total_params = sum(p.numel() for p in self.model.parameters())
    trainable_params = sum(p.numel() for p in self.model.parameters() if p.requires_grad)

    # Estimate model size (float32)
    model_size_mb = total_params * 4 / (1024 * 1024)

    return {
        'total_parameters': total_params,
        'trainable_parameters': trainable_params,
        'model_size_mb': model_size_mb
    }

```

2. Accuracy Benchmarking:

```

def benchmark_on_standard_datasets(model_configs, datasets):
    """
    Benchmark models on standard datasets for fair comparison
    """
    results = {}

```

```

for dataset_name, dataset in datasets.items():
    print(f"Benchmarking on {dataset_name}")
    results[dataset_name] = {}

    for model_name, config in model_configs.items():
        print(f"    Testing {model_name}")

        # Load model
        model = load_model_from_config(config)

        # Fine-tune if needed
        if config.get('finetune', True):
            model = finetune_model(model, dataset['train'], config['finetune_params'])

        # Evaluate
        predictions = model.predict(dataset['test']['inputs'])
        metrics = evaluate_classification(
            dataset['test']['labels'],
            predictions
        )

        results[dataset_name][model_name] = metrics

    return results

# Standard NLP datasets for benchmarking
standard_datasets = {
    'imdb': load_dataset('imdb'),
    'sst2': load_dataset('glue', 'sst2'),
    'cola': load_dataset('glue', 'cola'),
    'mrpc': load_dataset('glue', 'mrpc')
}

```

3. Resource Usage Monitoring:

```

import GPUtil
import threading
import time

class ResourceMonitor:
    def __init__(self, monitoring_interval=0.1):
        self.monitoring_interval = monitoring_interval
        self.monitoring = False
        self.cpu_usage = []
        self.memory_usage = []
        self.gpu_usage = []
        self.gpu_memory = []

    def start_monitoring(self):
        self.monitoring = True
        self.monitor_thread = threading.Thread(target=self._monitor_resources)
        self.monitor_thread.start()

    def stop_monitoring(self):
        self.monitoring = False

```

```

self.monitor_thread.join()

return {
    'avg_cpu_percent': np.mean(self.cpu_usage),
    'max_cpu_percent': np.max(self.cpu_usage),
    'avg_memory_gb': np.mean(self.memory_usage),
    'max_memory_gb': np.max(self.memory_usage),
    'avg_gpu_percent': np.mean(self.gpu_usage) if self.gpu_usage else 0,
    'max_gpu_memory_gb': np.max(self.gpu_memory) if self.gpu_memory else 0
}

def _monitor_resources(self):
    while self.monitoring:
        # CPU and RAM
        self.cpu_usage.append(psutil.cpu_percent())
        self.memory_usage.append(psutil.virtual_memory().used / (1024**3))

        # GPU
        try:
            gpus = GPUtil.getGPUs()
            if gpus:
                self.gpu_usage.append(gpus[0].load * 100)
                self.gpu_memory.append(gpus[0].memoryUsed / 1024)
        except:
            pass

        time.sleep(self.monitoring_interval)

```

4. Automated Benchmarking Pipeline:

```

def run_comprehensive_benchmark(models_config, test_data):
    """
    Run complete benchmarking pipeline
    """
    benchmark_results = {}

    for model_name, config in models_config.items():
        print(f"Benchmarking {model_name}...")

        # Load model
        model = load_model_from_config(config)
        benchmarker = ModelBenchmark(model)

        # Start resource monitoring
        monitor = ResourceMonitor()
        monitor.start_monitoring()

        # Run benchmarks
        model_results = {}

        # 1. Model size
        model_results['size_metrics'] = benchmarker.profile_model_size()

        # 2. Inference benchmark
        inference_metrics = benchmarker.benchmark_inference(test_data)

```

```

model_results['inference_metrics'] = inference_metrics

# 3. Accuracy metrics
predictions = model.predict(test_data)
accuracy_metrics = evaluate_model_accuracy(predictions, test_labels)
model_results['accuracy_metrics'] = accuracy_metrics

# Stop monitoring and get resource usage
resource_metrics = monitor.stop_monitoring()
model_results['resource_metrics'] = resource_metrics

benchmark_results[model_name] = model_results

# Generate comparison report
generate_benchmark_report(benchmark_results)

return benchmark_results

```

Deployment & Optimization

Q20: How do you optimize pretrained models for production deployment?

Answer:

Model Optimization Pipeline:

1. Model Quantization:

```

import torch
import torch.quantization

def quantize_model(model, calibration_data):
    """
    Post-training quantization for inference speedup
    """
    # Prepare model for quantization
    model.eval()
    model.qconfig = torch.quantization.get_default_qconfig('fbgemm')
    torch.quantization.prepare(model, inplace=True)

    # Calibration
    with torch.no_grad():
        for data, _ in calibration_data:
            model(data)

    # Convert to quantized model
    quantized_model = torch.quantization.convert(model, inplace=False)

    return quantized_model

# TensorFlow Lite quantization
def quantize_tf_model(model_path, representative_dataset):
    import tensorflow as tf

```

```

converter = tf.lite.TFLiteConverter.from_saved_model(model_path)

# Dynamic range quantization
converter.optimizations = [tf.lite.Optimize.DEFAULT]

# Full integer quantization
def representative_dataset_gen():
    for data in representative_dataset:
        yield [data.astype(np.float32)]

converter.representative_dataset = representative_dataset_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

tflite_model = converter.convert()
return tflite_model

```

2. Model Pruning:

```

def prune_model(model, pruning_ratio=0.5):
    """
    Remove less important weights to reduce model size
    """
    import torch.nn.utils.prune as prune

    parameters_to_prune = []
    for module in model.modules():
        if isinstance(module, torch.nn.Linear) or isinstance(module, torch.nn.Conv2d):
            parameters_to_prune.append((module, 'weight'))

    # Global magnitude pruning
    prune.global_unstructured(
        parameters_to_prune,
        pruning_method=prune.L1Unstructured,
        amount=pruning_ratio
    )

    # Remove pruning reparameterization
    for module, param_name in parameters_to_prune:
        prune.remove(module, param_name)

    return model

# Structured pruning (remove entire channels/neurons)
def structured_pruning(model, pruning_ratio=0.3):
    for module in model.modules():
        if isinstance(module, torch.nn.Conv2d):
            prune.ln_structured(
                module, name='weight', amount=pruning_ratio,
                n=2, dim=0 # Prune output channels
            )

```

3. Knowledge Distillation:

```
class DistillationTrainer:
    def __init__(self, teacher_model, student_model, temperature=3.0, alpha=0.7):
        self.teacher = teacher_model
        self.student = student_model
        self.temperature = temperature
        self.alpha = alpha

        self.teacher.eval() # Teacher always in eval mode

    def distillation_loss(self, student_logits, teacher_logits, true_labels):
        # Soft targets from teacher
        teacher_probs = torch.softmax(teacher_logits / self.temperature, dim=1)
        student_log_probs = torch.log_softmax(student_logits / self.temperature, dim=1)

        # KL divergence loss
        distill_loss = torch.nn.functional.kl_div(
            student_log_probs, teacher_probs, reduction='batchmean'
        ) * (self.temperature ** 2)

        # Standard cross-entropy loss
        ce_loss = torch.nn.functional.cross_entropy(student_logits, true_labels)

        # Combined loss
        total_loss = self.alpha * distill_loss + (1 - self.alpha) * ce_loss
        return total_loss

    def train_student(self, train_loader, num_epochs=10):
        optimizer = torch.optim.Adam(self.student.parameters(), lr=1e-3)

        for epoch in range(num_epochs):
            for batch_idx, (data, target) in enumerate(train_loader):
                # Get teacher predictions
                with torch.no_grad():
                    teacher_logits = self.teacher(data)

                # Get student predictions
                student_logits = self.student(data)

                # Calculate distillation loss
                loss = self.distillation_loss(student_logits, teacher_logits, target)

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
```

4. Model Compression and Deployment:

```
class ModelDeploymentOptimizer:
    def __init__(self, model, target_platform='cpu'):
        self.model = model
        self.target_platform = target_platform
```

```

def optimize_for_inference(self):
    """Apply various optimizations for inference"""
    optimized_model = self.model

    # 1. Set to evaluation mode
    optimized_model.eval()

    # 2. Fuse operations
    if hasattr(torch.quantization, 'fuse_modules'):
        optimized_model = self.fuse_conv_bn_relu(optimized_model)

    # 3. Convert to TorchScript
    optimized_model = self.convert_to_torchscript(optimized_model)

    # 4. Apply quantization if needed
    if self.target_platform in ['mobile', 'edge']:
        optimized_model = self.apply_quantization(optimized_model)

    return optimized_model

def fuse_conv_bn_relu(self, model):
    """Fuse Conv-BatchNorm-ReLU patterns"""
    modules_to_fuse = []
    for name, module in model.named_modules():
        if isinstance(module, torch.nn.Conv2d):
            # Look for Conv-BN-ReLU pattern
            modules_to_fuse.append([f'{name}', f'{name}_bn', f'{name}_relu'])

    if modules_to_fuse:
        model = torch.quantization.fuse_modules(model, modules_to_fuse)

    return model

def convert_to_torchscript(self, model):
    """Convert model to TorchScript for deployment"""
    # Trace the model
    dummy_input = torch.randn(1, 3, 224, 224) # Adjust size as needed
    traced_model = torch.jit.trace(model, dummy_input)

    # Optimize for inference
    traced_model = torch.jit.optimize_for_inference(traced_model)

    return traced_model

def export_to_onnx(self, input_shape, output_path):
    """Export model to ONNX format"""
    dummy_input = torch.randn(*input_shape)

    torch.onnx.export(
        self.model,
        dummy_input,
        output_path,
        export_params=True,
        opset_version=11,
        do_constant_folding=True,
        input_names=['input'],

```



```

        output_names=['output'],
        dynamic_axes={
            'input': {0: 'batch_size'},
            'output': {0: 'batch_size'}
        }
    )

```

5. Production Serving:

```

from fastapi import FastAPI
import torch
import asyncio

class ModelServer:
    def __init__(self, model_path, device='cpu'):
        self.model = torch.jit.load(model_path, map_location=device)
        self.model.eval()
        self.device = device

    async def predict(self, input_data):
        """Async prediction for better throughput"""
        with torch.no_grad():
            input_tensor = torch.tensor(input_data).to(self.device)
            output = self.model(input_tensor)
            return output.cpu().numpy()

    def batch_predict(self, batch_inputs, batch_size=32):
        """Batch processing for efficiency"""
        results = []

        for i in range(0, len(batch_inputs), batch_size):
            batch = batch_inputs[i:i+batch_size]
            batch_tensor = torch.tensor(batch).to(self.device)

            with torch.no_grad():
                batch_output = self.model(batch_tensor)
                results.extend(batch_output.cpu().numpy())

        return results

# FastAPI deployment
app = FastAPI()
model_server = ModelServer('optimized_model.pt')

@app.post("/predict")
async def predict(data: dict):
    result = await model_server.predict(data['input'])
    return {"prediction": result.tolist()}

@app.post("/batch_predict")
async def batch_predict(data: dict):
    results = model_server.batch_predict(data['inputs'])
    return {"predictions": results}

```

Ethics & Bias

Q21: What are the ethical considerations when using pretrained models?

Answer:

Key Ethical Concerns:

1. Bias in Pretrained Models:

```
def detect_bias_in_model(model, tokenizer, test_sentences):
    """
    Detect potential bias in language models
    """
    bias_results = {}

    # Gender bias test
    gender_templates = [
        "The {profession} said {pronoun} was qualified for the job.",
        "The {profession} explained that {pronoun} had extensive experience."
    ]

    professions = ['engineer', 'nurse', 'teacher', 'CEO', 'secretary']
    pronouns = {'male': ['he', 'his'], 'female': ['she', 'her']}

    for profession in professions:
        profession_bias = {}

        for gender, pronoun_list in pronouns.items():
            scores = []
            for template in gender_templates:
                for pronoun in pronoun_list:
                    sentence = template.format(profession=profession, pronoun=pronoun)

                    # Get model probability for sentence
                    inputs = tokenizer(sentence, return_tensors='pt')
                    with torch.no_grad():
                        outputs = model(**inputs)
                    # Calculate perplexity or likelihood
                    score = calculate_sentence_score(outputs, inputs)
                    scores.append(score)

            profession_bias[gender] = np.mean(scores)

        bias_results[profession] = profession_bias

    return bias_results

def calculate_sentence_score(outputs, inputs):
    """Calculate sentence likelihood score"""
    logits = outputs.logits
    labels = inputs.input_ids[:, 1:] # Shift for next token prediction
    logits = logits[:, :-1] # Remove last token logits

    # Calculate cross-entropy loss (lower = more likely)
```

```

loss = torch.nn.functional.cross_entropy(
    logits.reshape(-1, logits.size(-1)),
    labels.reshape(-1),
    reduction='mean'
)

return -loss.item() # Return negative loss as score

```

2. Bias Mitigation Strategies:

```

class BiasAwareFineTuning:
    def __init__(self, model, fairness_constraints=None):
        self.model = model
        self.fairness_constraints = fairness_constraints or {}

    def debias_training(self, train_data, protected_attributes):
        """
        Train model with fairness constraints
        """
        optimizer = torch.optim.AdamW(self.model.parameters())

        for epoch in range(self.num_epochs):
            for batch in train_data:
                # Standard loss
                outputs = self.model(batch['input'])
                task_loss = self.calculate_task_loss(outputs, batch['labels'])

                # Fairness loss
                fairness_loss = self.calculate_fairness_loss(
                    outputs, batch['protected_attrs']
                )

                # Combined loss
                total_loss = task_loss + self.fairness_lambda * fairness_loss

                optimizer.zero_grad()
                total_loss.backward()
                optimizer.step()

    def calculate_fairness_loss(self, predictions, protected_attrs):
        """
        Calculate fairness constraint violations
        """
        # Demographic parity:  $P(Y=1|A=0) = P(Y=1|A=1)$ 
        group_0_pred = predictions[protected_attrs == 0]
        group_1_pred = predictions[protected_attrs == 1]

        group_0_rate = torch.mean(torch.sigmoid(group_0_pred))
        group_1_rate = torch.mean(torch.sigmoid(group_1_pred))

        # Penalize difference in positive prediction rates
        fairness_loss = torch.abs(group_0_rate - group_1_rate)

        return fairness_loss

```

```

# Data augmentation for bias reduction
def create_balanced_dataset(original_data, protected_attribute):
    """
    Create more balanced dataset across protected attributes
    """
    balanced_data = []

    # Group data by protected attribute
    groups = {}
    for item in original_data:
        attr_value = item[protected_attribute]
        if attr_value not in groups:
            groups[attr_value] = []
        groups[attr_value].append(item)

    # Find minimum group size
    min_size = min(len(group) for group in groups.values())

    # Sample equally from each group
    for group in groups.values():
        sampled = random.sample(group, min_size)
        balanced_data.extend(sampled)

    return balanced_data

```

3. Responsible AI Practices:

```

class ResponsibleAIFramework:
    def __init__(self):
        self.audit_results = {}

    def conduct_model_audit(self, model, test_data):
        """
        Comprehensive model audit for responsible AI
        """
        audit_report = {}

        # 1. Performance across demographics
        audit_report['demographic_performance'] = self.analyze_demographic_performance(
            model, test_data
        )

        # 2. Robustness testing
        audit_report['robustness'] = self.test_model_robustness(model, test_data)

        # 3. Privacy assessment
        audit_report['privacy'] = self.assess_privacy_risks(model, test_data)

        # 4. Explainability analysis
        audit_report['explainability'] = self.analyze_model_explainability(model)

        return audit_report

    def analyze_demographic_performance(self, model, test_data):
        """Analyze performance across different demographic groups"""

```

```

demographic_results = {}

for group_name, group_data in test_data.items():
    predictions = model.predict(group_data['inputs'])

    metrics = {
        'accuracy': accuracy_score(group_data['labels'], predictions),
        'precision': precision_score(group_data['labels'], predictions, average='weighted'),
        'recall': recall_score(group_data['labels'], predictions, average='weighted'),
        'f1': f1_score(group_data['labels'], predictions, average='weighted')
    }

    demographic_results[group_name] = metrics

# Calculate fairness metrics
fairness_metrics = self.calculate_fairness_metrics(demographic_results)
demographic_results['fairness_metrics'] = fairness_metrics

return demographic_results

def calculate_fairness_metrics(self, demographic_results):
    """Calculate standard fairness metrics"""
    groups = list(demographic_results.keys())
    metrics = ['accuracy', 'precision', 'recall', 'f1']

    fairness_metrics = {}

    for metric in metrics:
        values = [demographic_results[group][metric] for group in groups]
        fairness_metrics[f'{metric}_difference'] = max(values) - min(values)
        fairness_metrics[f'{metric}_ratio'] = min(values) / max(values) if max(values) != 0 else 0

    return fairness_metrics

```

4. Documentation and Transparency:

```

class ModelCard:
    """
    Document model capabilities, limitations, and ethical considerations
    """
    def __init__(self, model_name):
        self.model_name = model_name
        self.model_details = {}
        self.intended_use = {}
        self.factors = {}
        self.metrics = {}
        self.evaluation_data = {}
        self.training_data = {}
        self.quantitative_analyses = {}
        self.ethical_considerations = {}
        self.caveats_recommendations = {}

    def generate_card(self):
        """Generate comprehensive model card"""
        card = {

```

```

        'model_details': {
            'model_name': self.model_name,
            'model_date': datetime.now().isoformat(),
            'model_version': self.model_details.get('version', '1.0'),
            'model_type': self.model_details.get('type', 'Unknown'),
            'paper_or_resources': self.model_details.get('resources', []),
            'license': self.model_details.get('license', 'Unknown')
        },
        'intended_use': {
            'primary_intended_uses': self.intended_use.get('primary_uses', []),
            'primary_intended_users': self.intended_use.get('primary_users', []),
            'out_of_scope_uses': self.intended_use.get('out_of_scope', [])
        },
        'factors': {
            'relevant_factors': self.factors.get('relevant', []),
            'evaluation_factors': self.factors.get('evaluation', [])
        },
        'metrics': {
            'performance_measures': self.metrics.get('performance', {}),
            'decision_thresholds': self.metrics.get('thresholds', {})
        },
        'evaluation_data': self.evaluation_data,
        'training_data': self.training_data,
        'quantitative_analyses': self.quantitative_analyses,
        'ethical_considerations': self.ethical_considerations,
        'caveats_and_recommendations': self.caveats_recommendations
    }

    return card

```

Practical Implementation

Q22: How do you implement a complete pipeline using pretrained models?

Answer:

End-to-End Pipeline Implementation:

1. Pipeline Architecture:

```

from abc import ABC, abstractmethod
import logging
from typing import Dict, List, Any

class PretrainedModelPipeline(ABC):
    """
    Base class for pretrained model pipelines
    """
    def __init__(self, config: Dict):
        self.config = config
        self.model = None
        self.preprocessor = None
        self.postprocessor = None

```

```

        self.logger = logging.getLogger(__name__)

    @abstractmethod
    def load_model(self):
        """Load pretrained model"""
        pass

    @abstractmethod
    def preprocess(self, input_data):
        """Preprocess input data"""
        pass

    @abstractmethod
    def predict(self, input_data):
        """Make predictions"""
        pass

    @abstractmethod
    def postprocess(self, predictions):
        """Postprocess predictions"""
        pass

    def __call__(self, input_data):
        """Run complete pipeline"""
        try:
            # Preprocess
            processed_input = self.preprocess(input_data)

            # Predict
            raw_predictions = self.predict(processed_input)

            # Postprocess
            final_predictions = self.postprocess(raw_predictions)

            return final_predictions

        except Exception as e:
            self.logger.error(f"Pipeline error: {str(e)}")
            raise

```

2. Text Classification Pipeline:

```

from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

class TextClassificationPipeline(PretrainedModelPipeline):
    def __init__(self, config: Dict):
        super().__init__(config)
        self.load_model()

    def load_model(self):
        """Load BERT-based classification model"""
        model_name = self.config['model_name']
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForSequenceClassification.from_pretrained(model_name)

```

```

# Load fine-tuned weights if available
if 'fine_tuned_path' in self.config:
    self.model.load_state_dict(torch.load(self.config['fine_tuned_path']))

self.model.eval()

# Move to GPU if available
device = 'cuda' if torch.cuda.is_available() else 'cpu'
self.model.to(device)
self.device = device

def preprocess(self, texts: List[str]):
    """Tokenize and prepare texts for model"""
    # Handle both single text and batch
    if isinstance(texts, str):
        texts = [texts]

    # Tokenize
    encoded = self.tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=self.config.get('max_length', 512),
        return_tensors='pt'
    )

    # Move to device
    encoded = {k: v.to(self.device) for k, v in encoded.items()}

    return encoded

def predict(self, processed_input):
    """Get model predictions"""
    with torch.no_grad():
        outputs = self.model(**processed_input)
        logits = outputs.logits

    # Get probabilities
    probabilities = torch.softmax(logits, dim=-1)

    return {
        'logits': logits.cpu().numpy(),
        'probabilities': probabilities.cpu().numpy()
    }

def postprocess(self, predictions):
    """Convert predictions to readable format"""
    probabilities = predictions['probabilities']

    results = []
    for probs in probabilities:
        predicted_class = int(probs.argmax())
        confidence = float(probs.max())

    # Map to label names if available

```



```

        label_names = self.config.get('label_names', {})
        predicted_label = label_names.get(predicted_class, str(predicted_class))

        results.append({
            'predicted_class': predicted_class,
            'predicted_label': predicted_label,
            'confidence': confidence,
            'all_probabilities': probs.tolist()
        })

    return results[0] if len(results) == 1 else results

# Usage example
config = {
    'model_name': 'bert-base-uncased',
    'max_length': 512,
    'label_names': {0: 'negative', 1: 'positive'}
}

pipeline = TextClassificationPipeline(config)
result = pipeline("This movie is amazing!")
print(result) # {'predicted_label': 'positive', 'confidence': 0.95, ...}

```

3. Image Classification Pipeline:

```

import torchvision.transforms as transforms
from torchvision import models
from PIL import Image

class ImageClassificationPipeline(PretrainedModelPipeline):
    def __init__(self, config: Dict):
        super().__init__(config)
        self.load_model()
        self.setup_transforms()

    def load_model(self):
        """Load pretrained CNN model"""
        model_name = self.config['model_name']

        if model_name == 'resnet50':
            self.model = models.resnet50(pretrained=True)
        elif model_name == 'efficientnet_b0':
            self.model = models.efficientnet_b0(pretrained=True)
        else:
            raise ValueError(f"Unsupported model: {model_name}")

        # Replace final layer if fine-tuned
        if 'num_classes' in self.config:
            num_classes = self.config['num_classes']
            if hasattr(self.model, 'fc'):
                self.model.fc = torch.nn.Linear(self.model.fc.in_features, num_classes)
            elif hasattr(self.model, 'classifier'):
                self.model.classifier[-1] = torch.nn.Linear(
                    self.model.classifier[-1].in_features, num_classes
                )

```

```

# Load fine-tuned weights
if 'fine_tuned_path' in self.config:
    self.model.load_state_dict(torch.load(self.config['fine_tuned_path']))

self.model.eval()

# Move to device
device = 'cuda' if torch.cuda.is_available() else 'cpu'
self.model.to(device)
self.device = device

def setup_transforms(self):
    """Setup image preprocessing transforms"""
    self.transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406], # ImageNet means
            std=[0.229, 0.224, 0.225]   # ImageNet stds
        )
    ])

def preprocess(self, images):
    """Preprocess images"""
    if not isinstance(images, list):
        images = [images]

    processed_images = []
    for img in images:
        # Handle different input types
        if isinstance(img, str):
            img = Image.open(img).convert('RGB')
        elif isinstance(img, np.ndarray):
            img = Image.fromarray(img).convert('RGB')

        # Apply transforms
        img_tensor = self.transform(img).unsqueeze(0)
        processed_images.append(img_tensor)

    # Batch images
    batch_tensor = torch.cat(processed_images, dim=0).to(self.device)

    return batch_tensor

def predict(self, processed_input):
    """Get model predictions"""
    with torch.no_grad():
        outputs = self.model(processed_input)
        probabilities = torch.softmax(outputs, dim=-1)

    return {
        'logits': outputs.cpu().numpy(),
        'probabilities': probabilities.cpu().numpy()
    }

```

```

def postprocess(self, predictions):
    """Convert predictions to readable format"""
    probabilities = predictions['probabilities']

    results = []
    for probs in probabilities:
        # Get top-k predictions
        top_k = self.config.get('top_k', 5)
        top_indices = probs.argsort()[-top_k:][::-1]

        top_predictions = []
        for idx in top_indices:
            class_name = self.config.get('class_names', {}).get(idx, f'class_{idx}')
            top_predictions.append({
                'class_id': int(idx),
                'class_name': class_name,
                'confidence': float(probs[idx])
            })

        results.append({
            'top_predictions': top_predictions,
            'predicted_class': int(probs.argmax()),
            'max_confidence': float(probs.max())
        })

    return results[0] if len(results) == 1 else results

```

4. Pipeline Factory:

```

class PipelineFactory:
    """Factory for creating different types of pipelines"""

    PIPELINE_TYPES = {
        'text_classification': TextClassificationPipeline,
        'image_classification': ImageClassificationPipeline,
        'question_answering': QuestionAnsweringPipeline,
        'text_generation': TextGenerationPipeline
    }

    @classmethod
    def create_pipeline(cls, pipeline_type: str, config: Dict):
        """Create pipeline instance"""
        if pipeline_type not in cls.PIPELINE_TYPES:
            raise ValueError(f"Unknown pipeline type: {pipeline_type}")

        pipeline_class = cls.PIPELINE_TYPES[pipeline_type]
        return pipeline_class(config)

    @classmethod
    def from_config_file(cls, config_path: str):
        """Create pipeline from configuration file"""
        import json

        with open(config_path, 'r') as f:
            config = json.load(f)

```

```

        pipeline_type = config.pop('pipeline_type')
        return cls.create_pipeline(pipeline_type, config)

# Configuration file example (config.json)
"""
{
    "pipeline_type": "text_classification",
    "model_name": "distilbert-base-uncased-finetuned-sst-2-english",
    "max_length": 512,
    "label_names": {"0": "NEGATIVE", "1": "POSITIVE"},
    "batch_size": 16
}
"""

# Usage
pipeline = PipelineFactory.from_config_file('config.json')
result = pipeline("This product is fantastic!")

```

Advanced Topics

Q23: What are the latest trends in pretrained models?

Answer:

Current Trends and Innovations:

1. Multimodal Models:

```

# Vision-Language Models (e.g., CLIP)
from transformers import CLIPProcessor, CLIPModel
from PIL import Image

class MultiModalPipeline:
    def __init__(self):
        self.model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
        self.processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

    def zero_shot_image_classification(self, image, candidate_labels):
        """Classify image using text descriptions"""
        # Prepare inputs
        inputs = self.processor(
            text=candidate_labels,
            images=image,
            return_tensors="pt",
            padding=True
        )

        # Get predictions
        outputs = self.model(**inputs)
        logits_per_image = outputs.logits_per_image
        probs = logits_per_image.softmax(dim=1)

```

```

# Return results
results = []
for i, label in enumerate(candidate_labels):
    results.append({
        'label': label,
        'score': float(probs[0][i])
    })

return sorted(results, key=lambda x: x['score'], reverse=True)

# Usage
pipeline = MultiModalPipeline()
image = Image.open("dog.jpg")
labels = ["a photo of a dog", "a photo of a cat", "a photo of a bird"]
results = pipeline.zero_shot_image_classification(image, labels)

```

2. Foundation Models and Emergent Capabilities:

```

# Large Language Models with emergent abilities
class FoundationModelAnalysis:
    def __init__(self):
        self.emergent_capabilities = {
            'few_shot_learning': self.analyze_few_shot,
            'chain_of_thought': self.analyze_cot,
            'in_context_learning': self.analyze_icl,
            'instruction_following': self.analyze_instruction_following
        }

    def analyze_few_shot(self, model, examples, test_case):
        """Analyze few-shot learning capability"""
        # Create few-shot prompt
        prompt = ""
        for example in examples:
            prompt += f"Q: {example['input']}\nA: {example['output']}\n\n"
        prompt += f"Q: {test_case}\nA: "

        # Generate response
        response = model.generate(prompt)
        return response

    def analyze_cot(self, model, question):
        """Analyze chain-of-thought reasoning"""
        cot_prompt = f"{question}\nLet's think step by step:"

        response = model.generate(cot_prompt)
        return response

    def scaling_law_analysis(self, model_sizes, performance_metrics):
        """Analyze how performance scales with model size"""
        import numpy as np
        from scipy.optimize import curve_fit

        def power_law(x, a, b):
            return a * (x ** b)

```

```

# Fit power law to scaling relationship
params, _ = curve_fit(power_law, model_sizes, performance_metrics)

return {
    'scaling_coefficient': params[0],
    'scaling_exponent': params[1],
    'predicted_performance': lambda size: power_law(size, *params)
}

```

3. Efficient Training and Inference:

```

# Parameter-Efficient Fine-tuning (PEFT) methods
class PEFTMethods:
    def __init__(self, base_model):
        self.base_model = base_model

    def apply_lora(self, rank=16, alpha=32):
        """Apply LoRA (Low-Rank Adaptation)"""
        from peft import LoraConfig, get_peft_model

        lora_config = LoraConfig(
            r=rank,
            lora_alpha=alpha,
            target_modules=["q_proj", "v_proj"],
            lora_dropout=0.1,
        )

        model = get_peft_model(self.base_model, lora_config)
        return model

    def apply_prefix_tuning(self, num_virtual_tokens=20):
        """Apply Prefix Tuning"""
        from peft import PrefixTuningConfig, get_peft_model

        prefix_config = PrefixTuningConfig(
            task_type="CAUSAL_LM",
            num_virtual_tokens=num_virtual_tokens,
        )

        model = get_peft_model(self.base_model, prefix_config)
        return model

    def apply_prompt_tuning(self, num_virtual_tokens=8):
        """Apply Prompt Tuning"""
        from peft import PromptTuningConfig, get_peft_model

        prompt_config = PromptTuningConfig(
            task_type="CAUSAL_LM",
            num_virtual_tokens=num_virtual_tokens,
        )

        model = get_peft_model(self.base_model, prompt_config)
        return model

```

4. Model Compression Innovations:

```
# Advanced quantization techniques
class AdvancedQuantization:
    def __init__(self, model):
        self.model = model

    def apply_qat(self):
        """Quantization Aware Training"""
        import torch.quantization

        # Prepare model for QAT
        self.model.qconfig = torch.quantization.get_default_qat_qconfig('fbgemm')
        torch.quantization.prepare_qat(self.model, inplace=True)

        return self.model

    def apply_mixed_precision(self):
        """Mixed precision training"""
        from torch.cuda.amp import GradScaler, autocast

        scaler = GradScaler()

        def train_step(data, target):
            with autocast():
                output = self.model(data)
                loss = loss_fn(output, target)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

        return train_step

    def apply_dynamic_quantization(self):
        """Dynamic quantization for inference"""
        quantized_model = torch.quantization.quantize_dynamic(
            self.model,
            {torch.nn.Linear, torch.nn.LSTM, torch.nn.GRU},
            dtype=torch.qint8
        )
        return quantized_model
```

5. AutoML for Pretrained Models:

```
class AutoMLPretrainedModels:
    def __init__(self):
        self.search_space = {
            'model_architectures': [
                'bert-base-uncased', 'roberta-base', 'distilbert-base-uncased'
            ],
            'learning_rates': [1e-5, 2e-5, 5e-5],
            'batch_sizes': [16, 32, 64],
            'fine_tuning_strategies': ['full', 'layer_wise', 'lora']
        }
```

```

}

def auto_model_selection(self, dataset, time_budget=3600):
    """Automatically select best model configuration"""
    import optuna

    def objective(trial):
        # Sample configuration
        model_name = trial.suggest_categorical('model', self.search_space['model_arch'])
        lr = trial.suggest_categorical('lr', self.search_space['learning_rates'])
        batch_size = trial.suggest_categorical('batch_size', self.search_space['batch_size'])

        # Train and evaluate model
        model = self.create_model(model_name)
        trainer = self.setup_trainer(model, lr, batch_size)

        # Quick evaluation (few epochs)
        results = trainer.train_and_evaluate(dataset, epochs=2)

        return results['validation_accuracy']

    # Run optimization
    study = optuna.create_study(direction='maximize')
    study.optimize(objective, timeout=time_budget)

    return study.best_params, study.best_value

def neural_architecture_search(self, base_architecture):
    """Search for optimal architecture modifications"""
    # Define search space for architecture modifications
    search_space = {
        'hidden_sizes': [256, 512, 768, 1024],
        'num_layers': [6, 8, 10, 12],
        'attention_heads': [8, 12, 16],
        'dropout_rates': [0.1, 0.2, 0.3]
    }

    # Implement differentiable architecture search
    # or evolutionary search for discrete choices
    pass

```

Future Directions:

- **Unified Models:** Single models handling multiple modalities and tasks
- **Retrieval-Augmented Generation:** Combining parametric and non-parametric knowledge
- **Constitutional AI:** Models trained with human feedback and principles
- **Tool-Using AI:** Models that can use external tools and APIs
- **Continual Learning:** Models that can learn new tasks without forgetting old ones

Quick Reference Guide

Model Selection Checklist

```
selection_criteria = {
    'task_requirements': [
        'classification', 'generation', 'regression', 'structured_prediction'
    ],
    'performance_constraints': [
        'accuracy_threshold', 'latency_requirement', 'memory_limit'
    ],
    'data_characteristics': [
        'dataset_size', 'domain_similarity', 'language', 'modality'
    ],
    'deployment_constraints': [
        'edge_device', 'cloud_deployment', 'real_time', 'batch_processing'
    ]
}
```

Fine-tuning Best Practices

1. **Start Simple:** Feature extraction before full fine-tuning
2. **Lower Learning Rates:** 1e-5 to 5e-5 for pretrained layers
3. **Layer-wise LR:** Different rates for different layers
4. **Early Stopping:** Monitor validation metrics
5. **Progressive Unfreezing:** Gradual layer unfreezing
6. **Data Quality:** Clean, representative training data

Common Pitfalls to Avoid

1. **Overfitting:** Use validation sets and regularization
2. **Wrong Preprocessing:** Match pretraining preprocessing
3. **Inappropriate Model:** Choose domain-appropriate models
4. **Insufficient Fine-tuning:** Don't under-train new layers
5. **Ignoring Bias:** Test for fairness and bias issues

Deployment Optimization Steps

1. **Model Quantization:** Reduce precision
2. **Pruning:** Remove redundant parameters
3. **Knowledge Distillation:** Train smaller student models
4. **ONNX Export:** Cross-platform deployment
5. **Batch Processing:** Optimize throughput
6. **Caching:** Cache frequent predictions

This comprehensive Pretrained Models cheat sheet covers fundamental concepts through cutting-edge techniques. Stay updated with the latest model releases and continue experimenting with different architectures for your specific use cases.